

Programming Assignment #1

Apriori 알고리즘을 이용한 연관 규칙 마이닝

과목: Data Science (ITE4005) 학번: 2023036299 이름: 김진욱 2026.03.11
OS: Windows 10/11 Python: 3.12.10 라이브러리: 표준 라이브러리 (sys, itertools)

1 알고리즘 요약

1.1 Apriori 알고리즘 개요

Apriori 알고리즘은 트랜잭션 데이터베이스에서 빈발 항목 집합(Frequent Itemset)을 찾고, 이를 바탕으로 연관 규칙(Association Rule)을 생성하는 알고리즘이다. 핵심 원리는 **Apriori Principle**으로, “빈발하지 않은 항목 집합의 상위 집합은 절대 빈발할 수 없다”는 성질을 이용해 후보 집합을 효율적으로 가지치기한다.

1.2 전체 순서도

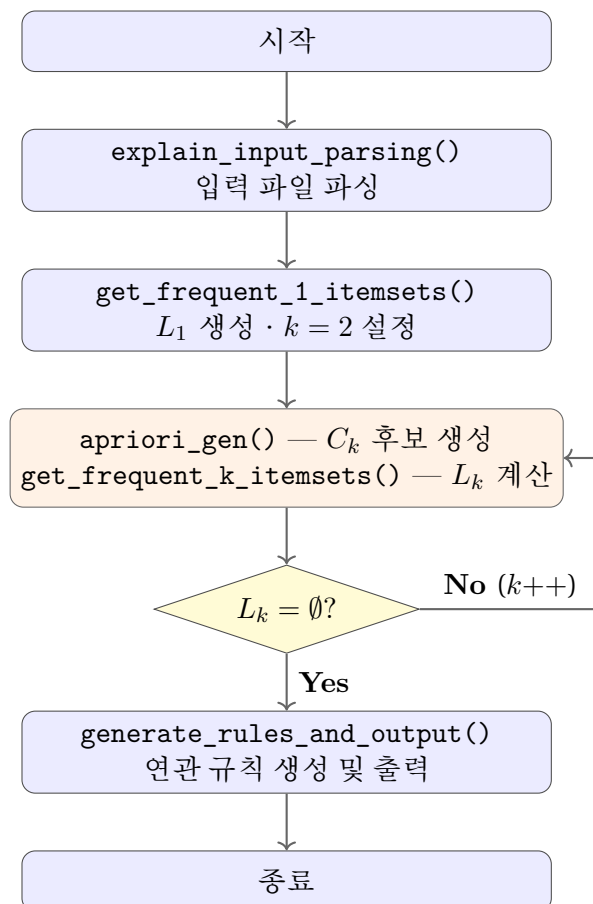


Figure 1: Apriori 알고리즘 전체 순서도

1.3 핵심 자료 흐름

L1(dict) -> keys -> apriori_gen -> C2(list) -> get_frequent_k -> L2(dict) -> keys ->

각 단계에서 빈발 항목 집합(L_k)은 딕셔너리 ({frozenset: 등장횟수}), 후보 집합(C_k)은 리스트 ([frozenset, ...])로 관리된다.

2 각 함수별 코드 상세 설명

2.1 explain_input_parsing(input_file) — 데이터 로드

입력 텍스트 파일을 읽어 트랜잭션 리스트로 변환한다.

```
1 def explain_input_parsing(input_file):
2     transactions = []
3     try:
4         with open(input_file, 'r', encoding='utf-8') as f:
5             for line_number, line in enumerate(f, start=1):
6                 items_str_list = line.strip().split()
7                 if not items_str_list:
8                     continue
9                 transaction = set(int(item) for item in items_str_list)
10                transactions.append(transaction)
11    except FileNotFoundError:
12        return None
13    return transactions
```

핵심 설계 결정:

- 각 트랜잭션을 set으로 저장한다. 이후 후보 집합의 포함 여부를 issuperset() 메서드로 $O(k)$ 에 판별하기 위함이다.
- FileNotFoundError 예외 처리로 파일이 없을 경우 None을 반환한다.

2.2 get_frequent_1_itemsets(transactions, min_sup_percent) — L_1 생성

길이가 1인 빈발 항목 집합을 생성한다.

```
1 def get_frequent_1_itemsets(transactions, min_sup_percent):
2     n_transactions = len(transactions)
3     min_count = n_transactions * (min_sup_percent / 100.0)
4
5     item_counts = {}
6     for transaction in transactions:
7         for item in transaction:
8             if item in item_counts:
9                 item_counts[item] += 1
```

```

10         else:
11             item_counts[item] = 1
12
13     L1 = {}
14     sorted_items = sorted(item_counts.items())
15     for item, count in sorted_items:
16         if count >= min_count:
17             itemset = frozenset({item})
18             L1[itemset] = count
19     return L1

```

핵심 설계 결정:

- frozenset 사용: 파이썬의 일반 set은 mutable이므로 딕셔너리의 키로 사용 불가. frozenset은 immutable이므로 키로 사용 가능하며, 이후 합집합 연산($\cup$) 시에도 자동으로 frozenset 타입이 유지된다.
- 딕셔너리 구조 ({frozenset: count}): 신뢰도(Confidence)를 계산할 때 `result[antecedent]`로 등장 횟수를 즉시 조회해야 하므로, 단순 리스트가 아닌 딕셔너리를 채택하였다.

2.3 apriori_gen(L_prev, k) — 후보 집합 C_k 생성

이전 단계의 빈발 항목 집합(L_{k-1})으로부터 길이 k 의 후보 집합을 생성한다.

```

1 def apriori_gen(L_prev, k):
2     Ck = []
3     len_L = len(L_prev)
4     for i in range(len_L):
5         for j in range(i+1, len_L):
6             list_i = list(L_prev[i])
7             list_j = list(L_prev[j])
8             list_i.sort()
9             list_j.sort()
10            if list_i[:k-2] == list_j[:k-2]:
11                candidate = L_prev[i] | L_prev[j]
12                if candidate not in Ck:
13                    Ck.append(candidate)
14    return Ck

```

핵심 설계 결정 — 최적화된 Join Step:

- 단순히 모든 쌍의 합집합을 구하고 길이를 검사하는 대신, 오름차순 정렬 후 앞쪽 $k-2$ 개 원소가 동일한 경우에만 합집합을 수행한다.
- $k-2$ 인 이유: 입력 원소의 길이가 $k-1$ 이고, 마지막 1개만 달라야 합집합 결과가 정확히 길이 k 가 되므로 비교 대상은 $(k-1) - 1 = k-2$ 개이다.
- 예시: $k = 3$ 일 때 $[1,2]$ 와 $[1,3]$ 은 앞의 1개($[:1] = [1]$)가 같으므로 합쳐서 $\{1,2,3\}$ 생성. 반면 $[1,2]$ 와 $[3,4]$ 는 앞이 다르므로 건너뛴다.

2.4 `get_frequent_k_itemsets(transactions, min_sup_percent, Ck)` — L_k 생성

후보 집합의 각 원소가 전체 트랜잭션에서 몇 번 등장하는지 세고, 최소 지지도를 넘는 것만 골라 L_k 를 만든다.

```
1 def get_frequent_k_itemsets(transactions, min_sup_percent, Ck):
2     n_transactions = len(transactions)
3     min_count = n_transactions * (min_sup_percent / 100.0)
4     candidate_counts = {candidate: 0 for candidate in Ck}
5     for transaction in transactions:
6         for candidate in Ck:
7             if candidate.issubset(transaction):
8                 candidate_counts[candidate] += 1
9     Lk = {}
10    for candidate, count in candidate_counts.items():
11        if count >= min_count:
12            Lk[candidate] = count
13    return Lk
```

핵심 설계 결정:

- `frozenset.issubset(set)` 메서드를 사용하여 후보 집합의 모든 원소가 해당 트랜잭션에 포함되는지를 효율적으로 검사한다.
- 지지도 기준(`min_count`)은 항상 전체 트랜잭션 수를 기준으로 계산한다 (후보 집합 수가 아님).

2.5 `find_freq(minimum_support, input_file)` — Apriori 메인 루프

L_1 부터 시작하여 L_k 가 빌 때까지 반복하며 모든 빈발 항목 집합을 하나의 딕셔너리에 모은다.

```
1 def find_freq(minimum_support, input_file):
2     result = {}
3     transactions = explain_input_parsing(input_file)
4     k = 1
5     previous = {}
6     while True:
7         Lk = {}
8         if k == 1:
9             Lk = get_frequent_1_itemsets(transactions, minimum_support)
10        else:
11            candidates = apriori_gen(list(previous.keys()), k)
12            Lk = get_frequent_k_itemsets(transactions, minimum_support,
13                                         ↪ candidates)
14        if len(Lk) == 0:
15            break
```

```

15         else:
16             result.update(Lk)
17             previous = Lk
18             k += 1
19     return result

```

핵심 설계 결정:

- `list(previous.keys())`: 딕셔너리의 키만 추출하여 리스트로 변환 후 `apriori_gen`에 전달한다. `apriori_gen`은 인덱스 기반 접근(`L_prev[i]`)을 사용하므로 반드시 리스트가 필요하다.
- `result.update(Lk)`: 각 단계의 L_k 딕셔너리를 하나의 `result` 딕셔너리에 병합한다. 이렇게 모든 길이의 빈발 집합과 등장 횟수를 한곳에 축적함으로써, 이후 연관 규칙 생성 시 추가 데이터베이스 스캔 없이 지지도/신뢰도를 계산할 수 있다.

2.6 `format_set(itemset)` — 출력 포매팅

`frozenset`을 과제 출력 포맷인 `{1,2,3}` 형태의 문자열로 변환한다.

```

1 def format_set(itemset):
2     str_itmes = [str(item) for item in sorted(itemset)]
3     joined_items = ",".join(str_itmes)
4     return f"{{{joined_items}}}"

```

- `sorted()`로 아이টে을 오름차순 정렬하여 출력 일관성을 보장한다.
- f-string의 이중 중괄호(`{{, }}`)를 활용하여 리터럴 `{}`를 출력한다.

2.7 `generate_rules_and_output(result, n_transactions, output_file)` — 연관 규칙 생성 및 파일 출력

모든 빈발 항목 집합에서 가능한 연관 규칙을 생성하고, 지지도/신뢰도를 계산하여 파일에 출력한다.

```

1 def generate_rules_and_output(result, n_transactions, output_file):
2     try:
3         with open(output_file, 'w', encoding='utf-8') as f:
4             for itemset in result.keys():
5                 length = len(itemset)
6                 if length < 2:
7                     continue
8                 for r in range(1, length):
9                     for subset in combinations(itemset, r):
10                         antecedent = frozenset(subset)
11                         consequent = itemset - antecedent
12                         sup = (result[itemset] / n_transactions) * 100
13                         conf = (result[itemset] / result[antecedent]) * 100

```

```

14         ant_str = format_set(antecedent)
15         con_str = format_set(consequent)
16         line = f"{ant_str}\t{con_str}\t{sup:.2f}\t{conf:.2f}\n"
17         f.write(line)
18     except IOError:
19         print(f"Error: Could not write to file '{output_file}'")

```

핵심 설계 결정:

- `itertools.combinations(itemset, r)`: 빈발 항목 집합에서 길이 r 인 모든 부분 집합을 생성한다. 이 부분집합이 **조건절(antecedent)**이 되고, 나머지(`itemset - antecedent`)가 **결과절(consequent)**이 된다.
- **지지도 (Support)**: $\frac{\text{result}[\text{itemset}]}{N} \times 100$ — 전체 트랜잭션 대비 해당 집합의 등장 비율.
- **신뢰도 (Confidence)**: $\frac{\text{result}[\text{itemset}]}{\text{result}[\text{antecedent}]} \times 100$ — 조건절을 포함하는 트랜잭션 중 전체 집합도 포함하는 비율 (조건부 확률).
- 동일한 아이템 쌍에서도 방향에 따라 신뢰도가 달라지므로 ($\{A\} \rightarrow \{B\} \neq \{B\} \rightarrow \{A\}$), 모든 방향의 규칙을 빠짐없이 생성한다.
- `:.2f` 포맷 지정자로 소수점 둘째 자리까지 반올림하여 과제 요구사항을 충족한다.

3 소스 코드 컴파일 및 실행 방법

3.1 실행 환경

- OS: Windows 10/11
- Python: 3.12.10
- 추가 라이브러리: 없음 (표준 라이브러리 `sys`, `itertools`만 사용)

3.2 실행 명령어

`python apriori.py [최소지지도(%)] [입력파일] [출력파일]`

3.3 실행 예시

`python apriori.py 5 input.txt output.txt`

위 명령어는 다음을 수행한다:

1. `input.txt`에서 트랜잭션 데이터를 읽는다.
2. 최소 지지도 5%를 기준으로 빈발 항목 집합을 찾는다.
3. 모든 연관 규칙을 생성하여 `output.txt`에 저장한다.

3.4 파일 구조

```
Data_Science_Assignment/  
├── apriori.py      <- 실행 파일 (소스 코드)  
├── input.txt       <- 입력 데이터  
└── output.txt      <- 출력 결과 (실행 후 자동 생성)
```

주의: 실행 파일, 입력 파일, 출력 파일은 반드시 같은 폴더에 위치해야 한다.

3.5 실행 화면

```
C:\Users\jwkim\Desktop\Data_Science_Assignment>python apriori.py 5 input.txt output.txt  
Running Apriori with min_sup=5.0% on input.txt...  
Done! Results saved to output.txt
```

Figure 2: 터미널에서 `python apriori.py 5 input.txt output.txt` 실행 화면

4 개발 과정 및 설계 결정 상세 기록

4.1 초기 접근 방식에 대한 재고

처음에는 Apriori 의사코드(pseudocode)를 그대로 구현하는 방식으로 접근하였으나, 단순 구현만으로는 각 단계의 의미를 충분히 파악하기 어렵다고 판단하였다. 이에 따라 의사코드의 각 단계를 하나씩 분해하여 점진적으로 구현하는 방식으로 전환하였다.

핵심 질문은 다음과 같았다:

“길이가 1인 원소를 담은 배열부터 시작하여, 특정 길이의 배열이 더 이상 생성되지 않을 때까지 해당 로직을 반복할 것인가?”

이 질문에 대한 답이 곧 Apriori 메인 루프(`find_freq`)의 구조가 되었다.

4.2 트랜잭션 데이터의 자료구조 선택

입력 파일의 각 줄(트랜잭션)을 어떤 자료구조로 저장할지 검토하였다.

자료구조	장점	단점
list	순서 보존, 인덱스 접근 가능	포함 여부 검사가 $O(n)$
set	<code>issubset()</code> 검사가 $O(k)$ 로 빠름	순서 없음

Table 1: 트랜잭션 자료구조 비교

결론: set 채택. Apriori 알고리즘에서 후보 집합의 포함 여부를 트랜잭션별로 반복 검사해야 하므로, `issubset()` 연산의 효율성이 결정적이었다.

```
1 # 최종 구현  
2 transaction = set(int(item) for item in items_str_list)
```

4.3 빈발 항목 집합(L_k) 저장 자료구조: list $\rightarrow$ dict

처음에는 L_1 을 단순 리스트로 구현하려 했으나, 이후 연관 규칙의 신뢰도 계산 시 각 집합의 등장 횟수(count)를 즉시 조회해야 한다는 점을 깨달았다.

```

1 # 초기 구현 (리스트) -- 등장 횟수를 알 수 없음
2 L1 = [frozenset({0}), frozenset({1}), ...]
3
4 # 최종 구현 (딕셔너리) -- 등장 횟수를 바로 조회 가능
5 L1 = {frozenset({0}): 134, frozenset({1}): 149, ...}

```

딕셔너리로 변경함으로써, 마지막 연관 규칙 생성 단계에서 `result[antecedent]`로 등장 횟수를 $O(1)$ 에 조회할 수 있게 되었다.

frozenset 도입 배경

딕셔너리의 키로 집합을 사용하려면 `immutable`(변경 불가능) 타입이어야 한다. 파이썬의 일반 `set`은 `mutable`이므로 키로 사용할 수 없다.

타입	mutable 여부	딕셔너리 키 사용	합집합 연산 결과
set	O (변경 가능)	불가	set
frozenset	X (변경 불가)	가능	frozenset

Table 2: set vs frozenset 비교

4.4 후보 집합(C_k)과 빈발 집합(L_k)의 자료구조 분리

구현 중 $C_k = \{\}$ (딕셔너리)로 초기화한 뒤 `Ck.append()`를 사용하여 문법 오류가 발생하였다. 이를 계기로 C_k 와 L_k 의 역할 차이를 명확히 정리하였다.

자료구조	역할	타입	이유
L_k	합격자 + 성적표	<code>dict{frozenset: int}</code>	등장 횟수를 키로 즉시 조회
C_k	후보자 명단	<code>list[frozenset]</code>	아직 카운팅 전, 순회에 적합

Table 3: L_k 와 C_k 자료구조 비교

전체 자료 흐름은 `L(dict)  $\rightarrow$  keys 추출  $\rightarrow$  apriori_gen  $\rightarrow$  C(list)  $\rightarrow$  get_frequent_k  $\rightarrow$  L(dict)` 순환 구조이다.

```

1 # 변경 전 (오류)
2 Ck = {} # 딕셔너리
3 Ck.append(...) # AttributeError
4
5 # 변경 후
6 Ck = [] # 리스트
7 Ck.append(...) # 정상 동작

```


4.5 후보 생성 최적화: 단순 합집합 $\rightarrow k-2$ 슬라이싱

초기 구현에서는 모든 쌍의 합집합을 구한 뒤 `len(candidate) == k`로 필터링하였다:

```
1 # 초기 구현 (단순 방식)
2 candidate = L_prev[i] | L_prev[j]
3 if len(candidate) == k and candidate not in Ck:
4     Ck.append(candidate)
```

이를 개선하기 위해 Apriori 논문의 **Join Step** 최적화를 도입하였다:

두 집합을 오름차순 정렬했을 때, 앞의 $k-2$ 개 원소가 동일하고 마지막 1개만 다를 때에만 합친다.

```
1 # 최종 구현 (최적화 방식)
2 list_i = list(L_prev[i])
3 list_j = list(L_prev[j])
4 list_i.sort()
5 list_j.sort()
6 if list_i[:k-2] == list_j[:k-2]:
7     candidate = L_prev[i] | L_prev[j]
8     if candidate not in Ck:
9         Ck.append(candidate)
```

$k-2$ 슬라이싱의 수학적 근거:

- 입력 원소의 길이 = $k-1$
- 마지막 1개만 달라야 합집합 길이가 정확히 k 가 됨
- 따라서 비교 대상 = $(k-1) - 1 = k-2$ 개

구체적 예시:

- $L_2 \rightarrow C_3$ ($k=3$): $[1,2]$ 와 $[1,3]$ 은 $[:1]$ 이 $[1]$ 로 같으므로 합격 $\rightarrow \{1,2,3\}$ 생성
- $L_3 \rightarrow C_4$ ($k=4$): $[1,2,3]$ 과 $[1,2,4]$ 는 $[:2]$ 가 $[1,2]$ 로 같으므로 합격 $\rightarrow \{1,2,3,4\}$ 생성
- $L_2 \rightarrow C_3$ ($k=3$): $[1,2]$ 와 $[3,4]$ 는 $[:1]$ 이 다름 $\rightarrow$ 건너뛰

4.6 메인 루프 구현 과정에서의 파이썬 문법 이슈

메인 루프(`find_freq`) 구현 과정에서 여러 파이썬 고유 문법 이슈를 발견하고 수정하였다.

(1) 빈 딕셔너리 검사: `is {}` → `len() == 0`

```
1 # 변경 전 (잘못된 방식 -- 무한루프 위험)
2 if (process is {}):
3     break
4
5 # 변경 후 (올바른 방식)
6 if len(Lk) == 0:
7     break
```

파이썬에서 `is` 연산자는 객체의 메모리 주소(identity)를 비교한다. 두 빈 딕셔너리는 동일한 값이지만 서로 다른 객체이므로 `is`로 비교하면 항상 `False`가 되어 무한루프에 빠진다.

(2) 딕셔너리 병합: `append()` → `update()`

```
1 # 변경 전 (오류)
2 result.append(process)
3
4 # 변경 후 (정상)
5 result.update(Lk)
```

`update()`는 인자로 받은 딕셔너리의 모든 키-값 쌍을 `result`에 병합한다.

(3) `apriori_gen` 인자 타입: `dict` → `list`

```
1 # 변경 전 (오류)
2 apriori_gen(previous, k)
3
4 # 변경 후 (정상)
5 apriori_gen(list(previous.keys()), k)
```

`apriori_gen`은 `L_prev[i]`와 같은 인덱스 기반 접근을 사용하므로, 딕셔너리를 그대로 전달하면 에러가 발생한다.

4.7 출력 포맷과 순서쌍(방향성) 논의

예시 출력 파일을 분석한 결과, 같은 아이템 쌍에 대해 두 줄이 출력됨을 발견하였다:

```
{1} {8} 15.40 51.68
{8} {1} 15.40 34.07
```

- 지지도(Support): 전체 트랜잭션 대비 확률이므로 순서 무관 → 둘 다 15.40%
- 신뢰도(Confidence): 조건부 확률이므로 순서에 따라 달라짐 → 분모(조건절의 등장 횟수)가 다르기 때문

4.8 f-string 포매팅 수정

파일 출력 시 f-string 문법 오류를 발견하고 수정하였다.

```
1 # 변경 전 (오류)
2 f.write(f"{format_set(antecedent)}+'\\t'+{format_set(consequent)}+'\\t'+...")
3
4 # 변경 후 (정상)
5 f.write(f"{format_set(antecedent)}\\t{format_set(consequent)}\\t{support:.2f}\\t{confidence:.2f}")
```

:.2f 포맷 지정자를 도입하여 소수점 둘째 자리 반올림 요구사항도 동시에 해결하였다.

4.9 함수 이름 및 변수명 가독성 리팩토링

코드 완성 후 가독성 향상을 위해 함수명과 변수명을 다음과 같이 리팩토링하였다.

변경 전	변경 후	변경 이유
main_function	find_freq	함수의 실제 역할(빈발 집합 찾기)을 반영
input (인자명)	input_file	파이썬 내장 함수 input() 과 이름 충돌 방지
output (인자)	삭제	함수 내에서 사용되지 않는 불필요한 인자
process (변수)	Lk	Apriori 논문 표기법과 일치시켜 의미 명확화

Table 4: 리팩토링 내역

4.10 각 함수별 자료구조 상세

각 함수에서 사용되는 핵심 자료구조를 정리하면 다음과 같다.

`generate_rules_and_output`에서의 규칙 생성 예시

빈발 집합 {1, 2, 3} 으로부터 생성되는 6가지 규칙:

5 테스트 결과

5.1 실행 조건

```
python apriori.py 5 input.txt output.txt
```

5.2 검증 결과

5.3 출력 샘플

2-아이템 규칙:

변수	타입	설명
<i>explain_input_parsing</i>		
transactions	list[set[int]]	전체 트랜잭션 목록
transaction	set[int]	개별 트랜잭션
<i>get_frequent_1_itemsets</i>		
item_counts	dict[int, int]	개별 아이템 → 등장 횟수
L1	dict[frozenset, int]	빈발 1-아이템셋 → 등장 횟수
<i>apriori_gen</i>		
L_prev	list[frozenset]	이전 단계 빈발 집합의 키 리스트
Ck	list[frozenset]	새로 생성된 후보 집합 리스트
list_i, list_j	list[int]	$k-2$ 비교를 위해 정렬된 임시 리스트
<i>get_frequent_k_itemsets</i>		
candidate_counts	dict[frozenset, int]	후보 → 등장 횟수 카운터
Lk	dict[frozenset, int]	합격한 빈발 k -아이템셋 → 등장 횟수
<i>find_freq</i>		
result	dict[frozenset, int]	모든 L_k 를 합친 최종 빈발 항목 집합
previous	dict[frozenset, int]	직전 단계의 L_k

Table 5: 함수별 핵심 자료구조 정리

조건절	결과절	지지도	신뢰도
{1}	{2, 3}	$\frac{\text{result}[\{1,2,3\}]}{N} \times 100$	$\frac{\text{result}[\{1,2,3\}]}{\text{result}[\{1\}]} \times 100$
{2}	{1, 3}	동일	$\frac{\text{result}[\{1,2,3\}]}{\text{result}[\{2\}]} \times 100$
{3}	{1, 2}	동일	$\frac{\text{result}[\{1,2,3\}]}{\text{result}[\{3\}]} \times 100$
{1, 2}	{3}	동일	$\frac{\text{result}[\{1,2,3\}]}{\text{result}[\{1,2\}]} \times 100$
{1, 3}	{2}	동일	$\frac{\text{result}[\{1,2,3\}]}{\text{result}[\{1,3\}]} \times 100$
{2, 3}	{1}	동일	$\frac{\text{result}[\{1,2,3\}]}{\text{result}[\{2,3\}]} \times 100$

Table 6: 빈발 집합 {1, 2, 3}에서의 규칙 생성 예시

{0}	{1}	6.60	24.63
{1}	{0}	6.60	22.15

→ 지지도 동일(6.60%), 신뢰도 상이(24.63% vs 22.15%) — 정상

3-아이템 규칙:

{0}	{8, 16}	6.60	24.63
{8}	{0, 16}	6.60	14.60
{16}	{0, 8}	6.60	15.57
{0, 8}	{16}	6.60	55.93
{0, 16}	{8}	6.60	64.71
{8, 16}	{0}	6.60	21.85

→ 빈발 집합 {0, 8, 16}에서 가능한 6가지 규칙 모두 생성 — 정상

검증 항목	결과
총 생성 규칙 수	1,066 개
출력 컬럼 수 (탭 구분)	모든 줄 4개 컬럼 ✓
중괄호 포맷 ({})	모든 아이탬셋 적용 ✓
소수점 둘째 자리 반올림	모든 지지도/신뢰도 XX.XX 형식 ✓
쌍방향 규칙 생성	$\{A\} \rightarrow \{B\}$ 와 $\{B\} \rightarrow \{A\}$ 모두 출력 ✓

Table 7: 출력 검증 결과